

Práctica 3 — WebRTC con servidor de señalización mediante WebSockets y extensiones de tiempo real

Contexto de la práctica

Esta práctica es una continuación de la **Práctica 1**, en la que se trabajó la base de una aplicación web cliente-servidor. En esta nueva práctica se amplía esa arquitectura incorporando comunicación multimedia en tiempo real entre navegadores. Para ello se utilizará **WebRTC**, una tecnología que permite transmitir audio, vídeo y datos directamente entre clientes.

Sin embargo, antes de que dos navegadores puedan comunicarse mediante WebRTC, necesitan intercambiar cierta información inicial. Este proceso se conoce como **señalización**.

En esta práctica, la señalización se realizará mediante **WebSockets**, utilizando preferentemente **Socket.IO** sobre Node.js y Express.

La idea principal es comprender la diferencia entre:

- comunicación cliente-servidor
- comunicación peer-to-peer
- señalización
- transmisión multimedia en tiempo real
- canales de datos y control
- extensiones avanzadas basadas en IA y otras tecnologías de tiempo real

Descripción general

El alumnado implementará una aplicación WebRTC mínima en la que dos navegadores puedan comunicarse mediante audio, vídeo y/o datos.

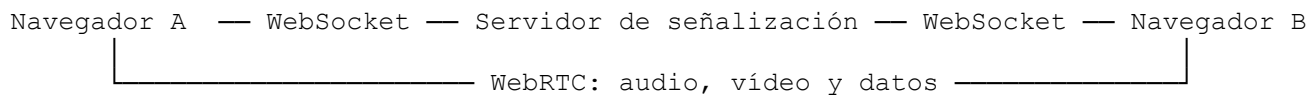
Para establecer la conexión WebRTC se utilizará un **servidor de señalización basado en WebSockets**. Este servidor será implementado preferentemente con **Socket.IO**, Node.js y Express.

El servidor deberá gestionar:

- usuarios conectados
- salas de comunicación
- entrada y salida de usuarios
- reenvío de mensajes `offer`

- reenvío de mensajes `answer`
- reenvío de candidatos ICE

Una vez establecida la conexión WebRTC, el flujo multimedia circulará directamente entre los navegadores.



Es importante destacar que el servidor de señalización **no transmite el audio ni el vídeo**. Su función es coordinar la conexión inicial.

Objetivo de la práctica

Implementar una aplicación mínima de comunicación WebRTC entre dos navegadores utilizando un servidor de señalización basado en WebSockets.

La aplicación debe permitir que dos clientes se conecten a una misma sala, intercambien la información necesaria para negociar una conexión WebRTC y establezcan una comunicación directa de audio, vídeo y/o datos.

Al finalizar la práctica, el alumnado deberá ser capaz de:

- reutilizar la arquitectura cliente-servidor trabajada en la Práctica 1
- crear un servidor básico con Node.js y Express
- utilizar WebSockets como canal de señalización
- emplear Socket.IO como abstracción de WebSocket
- gestionar usuarios y salas de comunicación
- capturar audio y vídeo desde el navegador
- crear una conexión WebRTC mediante `RTCPeerConnection`
- intercambiar mensajes `offer`, `answer` e ICE candidates
- visualizar vídeo local y vídeo remoto
- distinguir qué datos viajan por WebSocket y qué datos viajan por WebRTC
- utilizar `RTCDataChannel` para enviar datos o mensajes de control
- analizar posibles ampliaciones con tecnologías de tiempo real
- valorar la incorporación de IA online o IA local en navegador

Tecnologías utilizadas

La práctica utilizará las siguientes tecnologías principales:

- HTML
- CSS
- JavaScript
- Node.js
- Express

- WebSockets
- Socket.IO
- WebRTC
- `getUserMedia()`
- `RTCPeerConnection`
- `RTCDataChannel`

Además, como ampliación, se podrán incorporar o analizar otras tecnologías relacionadas con el tiempo real:

- WebTransport
- APIs de control remoto
- IA online
- Transformers.js
- modelos Transformer de Hugging Face

Tecnologías disponibles en la práctica

Tecnología	Tipo de comunicación	Uso en la práctica
WebSocket	cliente-servidor	señalización
Socket.IO	cliente-servidor	gestión de eventos, usuarios y salas
WebRTC	peer-to-peer	audio, vídeo y datos
RTCDataChannel	peer-to-peer	mensajes de control entre navegadores
WebTransport	cliente-servidor	extensión opcional para baja latencia
Transformers.js	cliente local	IA ejecutada en el navegador
IA online	cliente-servidor	análisis externo de audio, texto, vídeo o métricas

Qué datos no deben viajar por WebSocket

En esta práctica no deben viajar por WebSocket:

- vídeo en tiempo real
- audio en tiempo real
- flujo multimedia completo
- frames continuos de cámara
- datos intensivos que deban transmitirse directamente entre navegadores

Estos datos deben transmitirse mediante WebRTC.

Por tanto:

WebSocket
→ señalización y coordinación

WebRTC
→ audio, vídeo y datos multimedia

RTCDataChannel
→ datos directos entre peers

Extensiones de tiempo real

La práctica podrá ampliarse mediante tecnologías adicionales relacionadas con comunicación en tiempo real. Estas extensiones no sustituyen la parte obligatoria de WebRTC y WebSockets. Su finalidad es explorar otras posibilidades de interacción, control y análisis.

WebTransport puede analizarse como una tecnología cliente-servidor orientada a comunicaciones de baja latencia.

En esta práctica podría plantearse para:

- enviar telemetría al servidor
- enviar eventos rápidos de interacción
- registrar métricas de conexión
- comparar WebTransport con WebSocket
- estudiar escenarios de baja latencia cliente-servidor

Su uso será opcional y podrá plantearse como análisis arquitectónico o como prototipo experimental.

APIs de control remoto

La práctica también puede ampliarse con APIs relacionadas con control remoto o interacción externa.

Se podrán estudiar, entre otras:

- Remote Playback API
- Presentation API
- Gamepad API

Estas APIs pueden relacionarse con WebRTC mediante el envío de comandos o eventos a través de `RTCDataChannel`.

Ejemplos:

- usar un mando para silenciar el micrófono
- enviar un comando de presentación al navegador remoto
- mostrar información de la llamada en una pantalla externa
- controlar una interfaz compartida durante la llamada

IA

La práctica debe incorporar, como ampliación, funcionalidades basadas en IA online.

En este caso, la IA se utilizará como una capa adicional para analizar o interpretar información generada durante la sesión.

Posibles usos:

- transcripción de audio
- resumen de la sesión
- interpretación de comandos escritos o hablados
- análisis externo de texto, audio, vídeo o métricas

La IA online no sustituye a WebRTC ni al servidor de señalización.

Su papel es procesar información y devolver resultados útiles para la aplicación.

WebRTC

→ transmite audio, vídeo y datos

WebSocket

→ coordina la señalización

IA online

→ analiza información de la sesión

Transformers.js

Como alternativa a la IA online, se podrá analizar el uso de **Transformers.js** para ejecutar modelos de IA directamente en el navegador. Transformers.js permite trabajar con modelos preentrenados compatibles y ejecutar inferencia en cliente.

Posibles usos dentro de la práctica:

- análisis de texto
- clasificación de comandos
- generación de etiquetas
- detección de intención
- procesamiento local sin enviar datos al servidor
- apoyo a sistemas de control remoto

En este caso, el modelo se ejecuta en el navegador del usuario.

Transformers.js

→ IA local en cliente

IA online

→ IA ejecutada en un servicio externo o backend

El alumnado deberá diferenciar claramente ambas opciones.

Arquitectura entre WebRTC, WebSockets e IA

La arquitectura final puede entenderse de la siguiente manera:

Socket.IO / WebSocket

→ señalización: `join`, `offer`, `answer`, `ICE`

WebRTC

→ audio, vídeo y datos entre navegadores

RTCDataChannel

→ comandos y mensajes `peer-to-peer`

WebTransport

→ posible canal cliente-servidor de baja latencia

IA online

→ análisis externo de audio, texto, vídeo o métricas

Transformers.js

→ IA local ejecutada en el navegador

Requisitos mínimos de la práctica

La entrega mínima deberá incluir:

- aplicación web cliente-servidor basada en la Práctica 1
- servidor Node.js con Express
- canal de señalización mediante WebSockets o Socket.IO
- gestión básica de salas
- conexión de dos usuarios a la misma sala
- intercambio de mensajes `offer`
- intercambio de mensajes `answer`
- intercambio de candidatos ICE
- captura de cámara y/o micrófono
- visualización de vídeo local
- visualización de vídeo remoto
- explicación clara de qué datos viajan por WebSocket
- explicación clara de qué datos viajan por WebRTC
- integración con IA online y/o uso de Transformers.js en navegador

Memoria

La memoria deberá incluir:

- descripción general de la aplicación
- relación con la Práctica 1
- arquitectura del sistema
- explicación del servidor de señalización
- explicación del uso de WebSockets
- explicación del uso de WebRTC
- flujo `join` → `offer` → `answer` → `ICE`
- diferencia entre WebSocket y WebRTC
- descripción del uso de `RTCDataChannel`, si se utiliza
- uso e implementación de funcionalidades de IA

- problemas encontrados y soluciones aplicadas
- si se realiza una extensión, deberá documentarse indicando: tecnología utilizada y objetivo de la extensión